L I B R A R Y R E P L A C E M E N T A U D I T

Замена PyMuPDF на pypdfium2 в Stream-CAD

Аудит лицензионного риска AGPL: инвентаризация, карта API, патч замены и верификация эквивалентности

> База: репозиторий sod-project, ветка main, коммит cdb88c1
> Замена: PyMuPDF (AGPL) → pypdfium2 + reportlab + pypdf (BSD)
> Верификация: 529 тестов без PyMuPDF · дифференциальные прогоны
> Ноябрь 2026 г.

Содержание

1. Резюме.....1
2. Постановка задачи, исходные данные и ограничения.....2
3. Инвентаризация PyMuPDF на коммите cdb88c1.....2
3.1. Точки использования в рабочем коде.....2
3.2. Зависимости в requirements.txt.....3
3.3. Упоминания вне исполняемого кода.....3
4. Анализ функций и их заменимость.....3
4.1. Карта «API PyMuPDF → заменяющий стек».....3
4.2. Координатные системы: результаты зондирования.....4
4.3. Рендер, текст и геометрия.....4
4.4. Экспорт PDF с маркапами.....5
4.5. Генерация PDF (demo и тестовые фикстуры).....5
5. Реализация замены: состав патча.....5
5.1. Новый модуль-обёртка ai_checks/pdfium_tools.py.....5
5.2. Изменения в ИИ-контуре (ai_checks).....6
5.3. Изменения в storage и прочее.....6
6. Верификация эквивалентности.....6
6.1. Дифференциальные прогоны.....6
6.2. Полный тестовый прогон без PyMuPDF.....7
6.3. Верификация патча по протоколу HANDOFF №8.....7
7. Зона риска T66: векторное наложение версий.....7
8. Известные отличия и ограничения.....8
9. Лицензии и вывод.....8
10. Рекомендации по внедрению.....9

Примечание: содержание построено на полях Word. После редактирования документа кликните по содержанию правой кнопкой мыши и выберите «Обновить поле», чтобы актуализировать номера страниц.

1. Резюме

Задача аудита — подготовить и проверить план замены библиотеки PyMuPDF (AGPL-3.0, коммерческая лицензия Artifex) на стек с пермиссивными лицензиями в проекте Stream-CAD. Проведена полная инвентаризация использования PyMuPDF на коммите **cdb88c1** (ветка main), проанализирована заменимость каждого вызываемого API, написан единый git-патч по правилу HANDOFF №8 и выполнена дифференциальная верификация старой и новой реализации.

Главный вывод: полная замена возможна. Ни одна функция PyMuPDF не остаётся в продукте, покупка коммерческой лицензии Artifex не требуется. Заменяющий стек: pypdfium2 5.7.0 (BSD-3-Clause / Apache-2.0, ядро pdfium — BSD, Chromium), reportlab 4.4.9 и pypdf 6.6.0 (оба BSD-3-Clause). AGPL-риск для SaaS-модели Stream-CAD снимается полностью.

Эквивалентность новой реализации проверена тремя дифференциальными прогонами против PyMuPDF 1.26.7: правила нормоконтроля СПДС дают идентичные находки на всех 7 сценариях; рендер кропа совпадает попиксельно на 6 комбинациях поворота и сдвига MediaBox; позиции маркапов в экспорте PDF совпадают на 100% для поворотов 0/90/180/270. Полный тестовый прогон проекта — **529 тестов, все успешны, при полностью удалённом PyMuPDF**.

Патч затрагивает 17 файлов (+812/−285 строк): новый модуль-обёртку ai_checks/pdfium_tools.py, перевод ИИ-контура (правила Ф1, кропы, превью), перевод экспорта PDF с маркапами на reportlab+pypdf, генерацию фикстур тестов через reportlab и обновление requirements.txt. Зона риска T66 (векторное наложение версий) — планируемая функция, а не существующий код; после замены она реализуется на тех же примитивах pypdfium2 без AGPL.

2. Постановка задачи, исходные данные и ограничения

Поставщик задачи (роль аудитора) зафиксировал исходные условия: репозиторий github.com/ABEL678/sod-project, ветка main, коммит cdb88c1 «feat(ai_checks): выбор LLM-модели, RAG-синтез, UX-фиксы»; целевая библиотека — pypdfium2 (лицензия BSD); требуется найти все файлы с import fitz / PyMuPDF, оценить, какие функции используются (рендер в PNG, извлечение текста, координаты, кроп), написать патчи замены, а при невозможности полной замены — разделить функции на «заменяемые» и «оставляемые на PyMuPDF с покупкой лицензии Artifex».

Ограничения: не менять бизнес-логику (допустима только замена библиотеки с сохранением входов/выходов каждого сервиса); отдельно учесть, что PyMuPDF может использоваться для векторного наложения версий (T66) — это тоже зона AGPL-риска. Отчёт должен содержать список файлов, анализ функций, патчи и вывод.

Методика работы: (1) клонирование репозитория на точный коммит cdb88c1 и поиск всех вхождений; (2) чтение каждого затронутого файла с каталогизацией API; (3) зондирование — эмпирическая проверка координатных систем обеих библиотек на специально сгенерированных PDF (повороты 0/90/180/270, сдвинутый MediaBox); (4) написание заменяющего кода; (5) дифференциальные прогоны старого и нового кода на одинаковых входах; (6) полный тестовый прогон без PyMuPDF в окружении; (7) сборка и верификация git-патча.

3. Инвентаризация PyMuPDF на коммите cdb88c1

3.1. Точки использования в рабочем коде

Поиск по всем вхождениям import fitz, import pymupdf, строке «PyMuPDF» в коде и зависимостях выявил пять файлов рабочего кода и пять файлов тестов. Архивные материалы аудитов в docs/audit/* содержат исторические снимки кода и не являются исполняемыми — в объём патча они не входят.

Таблица 1 — Точки использования PyMuPDF в рабочем коде (cdb88c1)

Файл	API PyMuPDF	Назначение
ai_checks/domain/services.py	pymupdf.open; page.get_text(); page_count; page.rotation; page.mediabox; pymupdf.Rect; get_pixmap(clip, matrix); tobytes('jpeg')	Извлечение текста для RAG-чанков (Ф0.2), прогон правил (Ф1), пересчёт bbox при повороте листа, JPEG-кроп находки для Issue.crop_image
ai_checks/domain/rules.py	page.rect; page.number; get_drawings() ('l'/'re'); get_text('text', clip=); get_text('dict') (спаны: size, bbox); pymupdf.Rect	Детерминированные правила нормоконтроля SPDS.RAME (рамка), SPDS.STAMP (штамп), SPDS.FONT (минимальный кегль)
ai_checks/views.py	pymupdf.open; page.rect; pymupdf.Rect; page.draw_rect; get_pixmap(clip, Matrix(2,2)); tobytes('png')	PNG-превью находки с красной рамкой подсветки bbox (вьюх finding_preview)

Файл	API PyMuPDF	Назначение
storage/domain/services.py	fitz.open; page.rect; fitz.Rect/Point; draw_rect; draw_circle; insert_text; pdf.save(buffer)	Экспорт PDF текущей версии с наложенными маркапами замечаний (export_pdf_with_issues) — единственное место РЕДАКТИРОВАНИЯ содержимого PDF
storage/management/commands/create_demo_project.py	fitz.open() (пустой документ); new_page; insert_text; tobytes()	Генерация минимальных demo-PDF для демо-данных (dev-утилита)

Пять тестовых файлов ai_checks/tests/ (test_rules, test_run_rules, test_views, test_bbox_rotation, test_indexing) используют PyMuPDF для генерации фикстур (new_page, draw_rect, draw_line, insert_text, insert_textbox, save/tobytes) и передают объекты страниц PyMuPDF прямо в правила. Кроме того, pymupdf.open вызывается в шести местах, а исключение pymupdf.FileDataError обрабатывается через перехват Runtime Error в admin.py и views.py — это свойство необходимо сохранить в замене.

3.2. Зависимости в requirements.txt

В requirements.txt закреплены три пакета семейства PyMuPDF: pymupdf==2.0, pymupdf4llm==2.0 и pymupdf_layout==2.0. Поиск импортов по всему коду показал, что pymupdf4llm и pymupdf_layout нигде не используются — это мёртвые зависимости, установленные «про запас» для будущих RAG-сценариев. Они удаляются из requirements без какого-либо влияния на код.

3.3. Упоминания вне исполняемого кода

Текстовые упоминания PyMuPDF содержатся в комментариях и docstring: storage/domain/design_colors.py (RGB-константы «для PyMuPDF»), ai_checks/domain/models.py (docstring RuleDefinition), ai_checks/admin.py (комментарий о битых PDF), static/js/pdf_viewer.js (комментарий о координатной системе bbox) и генераторе дизайн-токенов. Они обновлены в патче, чтобы документация в коде не ссылалась на удалённую библиотеку.

4. Анализ функций и их заменимость

4.1. Карта «API PyMuPDF → заменяющий стек»

Все вызываемые API PyMuPDF разбиты на восемь функциональных групп. Для каждой указан заменяющий механизм и результат проверки эквивалентности. Ключевой вывод таблицы: критичных для замены пробелов у pypdfium2 нет;

единственная нетривиальная зона — редактирование PDF (маркапы) — закрывается связкой reportlab + pypdf.

Таблица 2 — Карта замены API

API PyMuPDF (использование)	Замена	Эквивалентность
pymupdf.open(); page.get_text()	pypdfium2: PdfDocument + PdfTextPage.get_text_range()	полная (нормализация \r\n → \n, завершающий \n)
get_text('text', clip=zone)	PdfTextPage.get_text_bounded(left, bottom, right, top)	полная, включая переносы строк
get_text('dict'): спаны (size, bbox, text)	посимвольно: FPDFText_GetFontSize + FPDFText_GetLooseCharBox + группировка в спаны по (базовая линия, кегль, шрифт)	полная для правил; bbox спанов ±0.01–1.5 pt
get_drawings(): items 'l' и 're'	FPDFPageObj (пути) + FPDFPath_GetPathSegment; 're' реконструируется из замкнутых подфигураций	полная (те же рёбра и прямоугольники)
page.rect; page.mediabox; page.rotation; page_count	FPDF_GetPageWidth/Height; FPDFPage_GetMediaBox; FPDFPage_GetRotation; len(PdfDocument)	полная (значения совпадают)
get_pixmap(clip, matrix) → PNG/JPEG	PdfPage.render(scale, rotation=0, crop=(l,b,r,t)) + PIL	попиксельно (допуск антиалиасинга ≤24/255)
draw_rect / draw_circle / insert_text / save	reportlab (оверлей-страница) + pypdf merge_page	попиксельно позиции маркапов (rot 0/90/180/270)
fitz.open() / new_page / insert_text (генерация PDF)	reportlab Canvas	полная для назначения (demo и фикстуры)
pymupdf.FileDataError (битый PDF)	PdfiumError — подкласс RuntimeError в обёртке	полная (перехват RuntimeError сохранён)

4.2. Координатные системы: результаты зондирования

Самая тонкая часть замены — координатные системы. Обе библиотеки используют несколько систем координат одновременно, и их соответствие неочевидно. Оно установлено эмпирически: на PDF с известной геометрией (рамка в прямоугольнике (100, 200, 300, 400), текст с базовой линией в известной точке) сравнивались показания обеих библиотек при поворотах страницы 0/90/180/270 и при сдвинутом MediaBox (начало в (100, 50)).

Таблица 3 — Соответствие координатных систем (проверено зондами)

API	Система координат
PyMuPDF page.rect	отображаемые размеры (с учётом /Rotate: для rot90 A4 — 842×595); начало TL, всегда (0, 0, w, h)
PyMuPDF get_drawings / get_text('dict')	НЕПОВЁРНУТЫЕ координаты, TL-начало (y вниз), относительно CropBox; от /Rotate НЕ зависят
PyMuPDF get_pixmap(clip=…)	clip интерпретируется в ОТОБРАЖАЕМОМ пространстве (проверено клипом x1=700 > ширины 595 неповёрнутого листа)
PyMuPDF draw_rect / insert_text на повернутом листе	контент пишется в поток НЕПОВЁРНУТЫМ (проверено обратным чтением get_drawings: прямоугольник (100,200,300,400) читается тем же) — подписи и маркапы на повернутых листах стоят в неповёрнутых позициях
pdfium: объекты и текст	неповёрнутое пользовательское пространство, BL-начало (y вверх), абсолютные координаты (включая начало MediaBox)
pypdfium2 render(rotation=0)	рендерит страницу КАК ОТОБРАЖАЕМУЮ (/Rotate применяется всегда); crop — поля (left, bottom, right, top) в пунктах относительно отображаемого листа

Из этих фактов выведены две рабочие формулы. Преобразование координат геометрии и текста (поворот страницы не влияет): **x_mu = x_user − cropbox.x0; y_mu = cropbox.y1 − y_user**. Преобразование кропа из координат page.rect в поля render(): **crop = (clip.x0 − rect.x0, rect.y1 − clip.y1, rect.x1 − clip.x1, clip.y0 − rect.y0)**. Обе формулы подтверждены попиксельным совпадением рендера на шести комбинациях поворота и сдвига MediaBox.

4.3. Рендер, текст и геометрия

Извлечение текста: pypdfium2 отдаёт последовательность символов с кеглем (FPDFText_GetFontSize), точными и «свободными» границами (GetCharBox / GetLooseCharBox) и базовой линией (GetCharOrigin). Свободные границы соответствуют bbox спана PyMuPDF с точностью до метрик шрифта (±0.01–1.5 pt). Группировка символов в «спаны» выполняется по непрерывной последовательности с одинаковой базовой линией, кеглем и именем шрифта — этого достаточно для правила SPDS.FONT, которое проверяет только кегль вне зоны штампа.

Векторная геометрия: пути pdfium раскрываются в рёбра через сегменты moveto/lineto/close; замкнутая подфигурация из четырёх углов с горизонтальными и вертикальными рёбрами распознаётся как прямоугольник (эквивалент item 're').

Прямоугольник из оператора 're' даёт те же четыре ребра, что и в PyMuPDF, поэтому счётчики линий правил SPDS.RAME и SPDS.STAMP совпадают.

4.4. Экспорт PDF с маркапами

PDFium — библиотека рендеринга и разбора: она не редактирует содержимое PDF. Функция export_pdf_with_issues переведена на схему «оверлей + слияние»: reportlab строит отдельную одностраничную PDF-подложку для каждой страницы исходника (рамка-прямоугольник с толщиной 2, окружность радиусом 9, подпись «#pk» шрифтом Helvetica 8 — те же параметры, что у PyMuPDF-отрисовки), затем pypdf клонирует исходный документ и вливает подложку в соответствующие страницы. Исходное содержимое страниц сохраняется без перекодирования.

Существенное зондирование: PyMuPDF на страницах с /Rotate ≠ 0 пишет маркапы неповёрнутыми, при этом координаты считает по отображаемым размерам page.rect. Новая реализация воспроизводит это поведение в точности (размеры под умножение ratios — отображаемые, позиция — неповёрнутая), поэтому даже квирк старой реализации сохранён. Попиксельное совпадение подтверждено для всех четырёх поворотов.

4.5. Генерация PDF (demo и тестовые фикстуры)

Генерация переведена на reportlab: demo-команда create_demo_project строит A4-страницу с текстом в точке, эквивалентной прежней (72, 72) в TL-системе; кириллица в base14-шрифте не поддерживалась и в PyMuPDF (извлекалась как «·····») — поведение сохранено, при желании владелец может зарегистрировать TTF. Для тестов написан общий помощник ai_checks/tests/pdfgen.py, принимающий координаты в прежней TL-системе PyMuPDF и конвертирующий их в BL-систему reportlab — фикстуры сохранили прежние координаты, поэтому правила проверяют те же места листа.

5. Реализация замены: состав патча

Патч stream-cad_pymupdf-to-pypdfium2.patch собран командой git diff --no-ext-diff --full-index --binary от базового коммита cdb88c1 и содержит 17 файлов: 15 изменённых и 2 новых, суммарно +812/−285 строк. Ниже — состав по областям кода.

5.1. Новый модуль-обёртка ai_checks/pdfium_tools.py

Единственная точка зависимости ИИ-контура от pypdfium2. Модуль документирует координатные системы, открывает PDF с переводом ошибок разбора в PdfiumError(RuntimeError) и даёт обёртку страницы PdfPageView с интерфейсом,

необходимым правилам: number, rect, rotation, mediabox, итераторы iter_line_segments(), iter_rects(), iter_text_chars(), iter_text_runs(), текст get_text(clip=…) и рендер render_clip() / render_jpeg_clip() (JPEG с качеством 95 — как у PyMuPDF tobytes('jpeg')). Класс Rect повторяет необходимый интерфейс pymupdf.Rect (x0/y0/x1/y1, width, height, is_empty), чтобы код правил оставался близким к прежнему.

5.2. Изменения в ИИ-контуре (ai_checks)

Таблица 4 — Изменения в ai_checks

Файл	Суть изменений
ai_checks/pdfium_tools.py	**НОВЫЙ**: обёртка pypdfium2 (см. 5.1)
ai_checks/domain/rules.py	Правила SPDS.RAME/STAMP/FONT переведены с объектов PyMuPDF на PdfPageView: линии — iter_line_segments, 're'-прямоугольники — iter_rects, текст зоны — get_text(clip), спаны шрифта — iter_text_runs. Пороги, зоны, тексты находок и формат {page, bbox, message, severity, evidence} не изменены; обёртка pymupdf_rect удалена
ai_checks/domain/services.py	extract_pages — постраничный get_text с нормализацией переносов; run_rules_check — обход страниц через PdfPageView с гарантированным close; bbox_ratios_for_rotation — page_count/rotation/mediabox из обёртки, вся математика пересчёта поворота сохранена дословно; crop_finding_image — построение clip из mediabox прежнее, рендер через render_jpeg_clip
ai_checks/views.py	finding_preview: рендер кропа через render_clip, красная рамка подсветки рисуется на растре средствами PIL (Position = (bbox − clip) × scale, цвет (217, 26, 26), толщина 1.2 pt × 2) — визуальный эквивалент прежней вписанной в PDF рамки; константы PREVIEW_SCALE=2 и BBOX_STROKE_PT=1.2 вынесены явно
ai_checks/tests/pdfgen.py	**НОВЫЙ**: генерация фикстур через reportlab в TL-координатах PyMuPDF (rects/lines/texts/wrapped_texts, многостраничность)
ai_checks/tests/test_*.py (5 файлов)	Фикстуры переведены на pdfgen; в test_rules.py make_page возвращает (doc, PdfPageView); все утверждения тестов сохранены дословно
ai_checks/admin.py, tests/base.py, domain/models.py	Обновлены комментарии (PdfiumError наследует RuntimeError; pypdfium2 в RuleDefinition; шрифтовая заметка)

5.3. Изменения в storage и прочее

Таблица 5 — Изменения в storage и инфраструктуре

Файл	Суть изменений
storage/domain/services.py	import fitz удалён; export_pdf_with_issues — оверлей reportlab + слияние pypdf (clone_from сохраняет исходные потоки); вспомогательные _display_to_user и _draw_issue_marker; поведение на повернутых листах воспроизведено точно (см. 4.4)
storage/management/commands/create_demo_project.py	_pdf — через reportlab Canvas (A4, Helvetica 14, точка вставки эквивалентна прежней)
requirements.txt	− pymupdf==2.0, − pymupdf4llm==2.0, − pymupdf_layout==2.0; + pypdfium2==5.7.0, + reportlab==4.4.9, + pypdf==6.6.0
static/js/pdf_viewer.js	Комментарий о координатной системе bbox переформулирован без привязки к PyMuPDF

6. Верификация эквивалентности

6.1. Дифференциальные прогоны

Дифференциальная верификация построена на принципе «одинаковый вход → одинаковый выход»: фикстуры генерируются PyMuPDF (как в реальной системе), затем старый и новый код обрабатывают одни и те же файлы, и результаты сравниваются поэлементно или попиксельно.

Таблица 6 — Дифференциальный прогон правил СПДС (7 сценариев)

Сценарий (фикстура тестов)	SPDS.RAME	SPDS.STAMP	SPDS.FONT
clean_sheet (рамка + штамп + текст 10 pt)	≡	≡	≡
bare_page (только текст)	≡	≡	≡
table_stamp (таблица тонких линий)	≡	≡	≡
small_font (кегль 5 pt)	≡	≡	≡ (bbox ±0.01 pt)
stamp_zone_text (мелкий текст в штампе)	≡	≡	≡
legacy_stamp_text («Основная надпись»)	≡	≡	≡

Сценарий (фикстура тестов)	SPDS.RAME	SPDS.STAMP	SPDS.FONT
gost_labels (поля ГОСТ-штампа)	≡	≡	≡

≡ — находки идентичны старому прогону: количество, severity, message, evidence; bbox — с точностью до 0.01 pt (различие метрик шрифта между библиотеками).

Таблица 7 — Дифференциальный прогон рендера и экспорта

Проверка	Кейсы	Результат
Рендер кропа: get_pixmap(clip) ↔ render(crop)	rot 0/90/180/270; сдвинутый Media Box; сдвиг+rot90	100% пикселей в допуске антиалиасинга ≤24/255; размеры и bbox контента совпадают во всех 6 кейсах
Экспорт с маркапами (рамка + окружность + подпись)	rot 0/90/180/270	маски пикселей цвета маркапов совпали на 100%: 6796/6796, 7469/7469, 6781/6781, 7466/7466 — расхождений 0
Извлечение текста и клип-текст зоны штампа	многострочный штамп («Стадия/Лист/Листов»)	строки идентичны (после \r\n → \n и завершающего \n)
Битый PDF (%PDF-1.4 broken)	1 кейс	pypdfium2 отказывается открывать; PdfiumError(RuntimeError) → Check Run.status=failed, как раньше

6.2. Полный тестовый прогон без PyMuPDF

Из окружения пакет PyMuPDF удалён полностью (pip uninstall pymupdf pymupdf4llm pymupdf-layout), после чего установлен requirements.txt патча и выполнен полный прогон: **python manage.py test → 529 тестов, все успешны** (единственный skip — тест PgVectorBackend-поиска, требующий PostgreSQL; в dev-среде это ожидаемое поведение, к замене отношения не имеет). Продолжительность прогона — около 380 секунд, сопоставима с прежней.

6.3. Верификация патча по протоколу HANDOFF №8

Патч проверен скриптом верификации на свежем клоне репозитория: (1) checkout cdb88c1; (2) git apply --check — без конфликтов; (3) применение патча; (4) пофайловое сравнение SHA-256 результата с эталонным коммитом — все 17 файлов байт-идентичны; (5) контроль состава — ровно 17 заголовков diff --git. Патч не

содержит миграций, бинарных файлов и изменений за пределами перечисленных файлов.

7. Зона риска T66: векторное наложение версий

Проверка подозрения «PyMuPDF используется для векторного наложения версий (T66)» дала однозначный результат: в текущем коде cdb88c1 серверного наложения НЕТ. Работающее «Сравнение / наложение версий» — клиентская «шторка» на pdf.js в static/js/pdf_viewer.js и не зависит от серверных библиотек. Серверное векторное наложение описано в MASTER_SPEC 18.4.1 как планируемое решение («бэкенд берет две страницы PDF, извлекает векторные пути drawings, окрашивает старую/новую версии») — именно поэтому PyMuPDF фигурировал в планах как будущая зависимость.

Вывод по T66: AGPL-риск для этой функции существовал только в плане. После замены она реализуется без PyMuPDF: примитивы уже опробованы в pdfium_tools (итератор путей и сегментов через FPDFPageObj/FPDFPath_GetPathSegment даёт ту же геометрию, что get_drawings). Требование к реализации T66: не возвращать PyMuPDF в стек; цветовой diff и генерация результата — на pypdfium2 + reportlab/pypdf.

8. Известные отличия и ограничения

Замена сохраняет бизнес-логику, но некоторые вторичные характеристики меняются. Полный перечень отличий с оценкой влияния приведён ниже — это осознанные компромиссы, а не дефекты.

Таблица 8 — Известные отличия новой реализации

Область	Отличие	Влияние
bbox текстовых спанов (правило SPDS.FONT, подсветка находок)	Loose box pdfium строится на метриках шрифта pdfium; расхождение с bbox PyMuPDF ±0.01–1.5 pt	Только геометрия рамки подсветки; критерий кегля и текст находки идентичны
Извлечённый текст страниц (RAG-чанки)	Переносы \r\n нормализуются в \n; непустая страница завершается \n как в PyMuPDF	Возможны сдвиги границ чанков на несколько символов; качество RAG не меняется

Область	Отличие	Влияние
Рендер	Движки mupdf и pdfium различаются антиалиасингом и хинтингом (в прогонах 100% пикселей в допуске ≤24/255)	Визуально эквивалентно; байтовое сравнение изображений не совпадает
Экспорт PDF с маркапами	pypdf сохраняет исходные потоки страниц и добавляет оверлей; PyMuPDF перекодировал документ целиком	Визуально идентично; байтовое представление файла другое (размер может отличаться)
Demo-PDF (create_demo_project)	base14 Helvetica не содержит кириллицы — как и прежний 'helv' в PyMuPDF (извлекалась как «·····»)	Косметика dev-данных; при желании регистрируется TTF с кириллицей
Группировка спанов шрифта	FPDFText_GetFontInfo вызывается на каждый символ; спаны группируются по (базовая линия, кегль, шрифт) — на смешанных строках гранулярность может отличаться от PyMuPDF	Для правила SPDS.FONT результат одинаков; на плотных листах при необходимости добавить кэш имён шрифтов
Скрытые слои (OCG) и Form XObjects	pdfium обходит Form XObjects (глубина 15); OCG-видимость не фильтруется — PyMuPDF get_drawings ведёт себя сопоставимо	Для рамки/штампа/кегля на реальных чертежах расхождений не выявлено

9. Лицензии и вывод

Таблица 9 — Лицензии PDF-стека до и после замены

Пакет	Лицензия	Статус	Комментарий
PyMuPDF	AGPL-3.0 / коммерческая Artifex	**УДАЛЁН**	Требовал AGPL-совместимого использования всего продукта либо лицензии Artifex
pypdfium2 5.7.0	BSD-3-Clause / Apache-2.0	**НОВЫЙ**	Ядро pdfium (Chromium) — BSD-3-Clause; бинарники встраиваются

Пакет	Лицензия	Статус	Комментарий
reportlab 4.4.9	BSD-3-Clause	**НОВЫЙ**	Open-source редакция; оверлеи маркапов и demo-PDF
pypdf 6.6.0	BSD-3-Clause	**НОВЫЙ**	Слияние оверлея с исходным документом

Ответ на ключевой вопрос задачи: полная замена возможна и выполнена. Функции, которые требовалось оставить на PyMuPDF с покупкой коммерческой лицензии Artifex, отсутствуют: рендер страницы в изображение (PIL/PNG), извлечение текста и координат, кроп области (Issue.crop_image), превью находок с подсветкой, правила нормоконтроля, экспорт с маркапами и генерация PDF переведены на pypdfium2 / reportlab / pypdf. Зона T66 остаётся планом и реализуется на том же пермиссивном стеке. AGPL-риск для SaaS-модели Stream-CAD снят полностью.

Дополнительная согласованность с ранее принятым планом OSS-обзора: pypdfium2 — один из поддерживаемых бэкендов разбора PDF в Docling, планируемом к внедрению, поэтому новая зависимость не добавляет разнобоя в стек, а pypdf и reportlab уже используются во вспомогательных ролях во многих Django-проектах.

10. Рекомендации по внедрению

Применение патча у владельца — по инструкции README_APPLY.md, идущей в поставке вместе с патчем: git apply на cdb88c1, затем pip uninstall pymupdf pymupdf4llm pymupdf-layout и pip install -r requirements.txt, затем полный python manage.py test (ожидается 529 OK) и контрольный python manage.py create_demo_project.

- **Шаг 1 (сразу):** применить патч, вычистить окружения (в том числе production-образ и docker-слои) от пакетов pymupdf*.
- **Шаг 2 (следующий docs-патч v1.7):** обновить ARCHITECTURE.md (описание export_pdf_with_issues, строки ~497 и ~669) и MASTER_SPEC 18.4.1 — убрать привязку T66 к PyMuPDF, зафиксировать стек pypdfium2 + reportlab + pypdf.
- **Шаг 3 (перед ФЗ):** прогнать правила и кропы на эталоне 30–50 реальных листов, включая повернутые (/Rotate ≠ 0) и сканы, зафиксировать статистику совпадений находок с приёмкой владельцем.

- **Шаг 4 (эксплуатация):** добавить в логи мониторинга LLMCallLog-независимую метрику времени extract_pages/run_rules_check; при плотной типографике больших PDF включить кэш имён шрифтов в iter_text_runs.
- **Шаг 5 (гигиена):** в CI запретить повторное появление зависимостей AGPL: grep-проверка requirements и импортов (import fitz / import pymupdf) на каждый merge.